

Recursion in SHACL

Where we stand, and what we can propose

Ognjen (Oggy) Savković • W3C Data Shapes WG

Introducing recursion into SHACL

It is **not easy**. Whatever one picks, something gives:

expressiveness
of the fragment

efficiency
of validation

simplicity
of implementation

I am looking for a **proposal** that

1. **is built on top of the validator every implementation already has;**
2. **is sound with respect to some well-defined recursive fragment.**

Two parts

Part I — Recursion in SHACL

The problem, what the community has settled, which fragment is safe.

Part II — What goes into the spec

The concrete text proposed for §6.5.3.

Background: nine years of community work on recursion semantics for SHACL — our first paper (ISWC 2018) up to the most recent one (KR 2026), which summarises all of it.

Full explanation in the **issue**: github.com/w3c/data-shapes/issues/1199

What validators actually do — SHACL

Separation tests (KR 2026) run against real validators; right of the double bar, four classical semantics.

Test	pySHACL	SHACL-S	Jena	TopBraid	rudof	GFP	LFP	bSMS	cSMS
bsep1	●	●	●	●	●	●	■	●	■
bsep2	●	●	●	●	●	●	■	●	■
bsep3	✗	✗	●	■	✗	●	■	●	■
bsep4	●	●	●	●	●	●	■	●	■
reach1	●	●	●	■	●	●	■	●	■
reach2	●	●	●	●	✗	■	●	●	■
safe1	●	●	●	●	✗	●	■	●	■
safe2	●	●	●	■	●	●	■	●	■

● pass ■ fail ✗ error

Five implementations, five different answers.

The control group: ShEx

Same tests, against ShEx — which **standardised** its recursion semantics.

Test	rudof (ShEx)	Jena ShEx	ShEx-S	GFP	LFP	bSMS	cSMS
bsep1	●	●	●	●	■	●	■
bsep2	●	●	●	●	■	●	■
bsep3	●	●	●	●	■	●	■
bsep4	●	●	●	●	■	●	■
reach1	●	●	●	●	■	●	■
reach2	■	■	■	■	●	●	■
safe1	●	●	●	●	■	●	■
safe2	●	●	●	●	■	●	■

Three implementations, **no disagreement, no errors**. Uniform behaviour.

Rich and KnowsRich: abstract shapes, real SHACL

Shapes

Rich

$\geq_1$ owns.yacht

KnowsRich

$\geq_1$ friend.**Rich**

```
ex:RichShape
  a sh:NodeShape ;
  sh:property [ sh:path ex:hasYacht ; sh:minCount 1 ] .
```

```
ex:KnowsRichShape
  a sh:NodeShape ;
  sh:targetNode ex:Tim ;
  sh:property [ sh:path ex:hasFriend ;
    sh:qualifiedValueShape ex:RichShape ;
    sh:qualifiedMinCount 1 ] .
```

Target

KnowsRich(Tim)?

For simplicity, we'll use the abstract syntax on the left instead of the SHACL on the right

Concept of Shape assignment (what a validator already computes)

Shapes

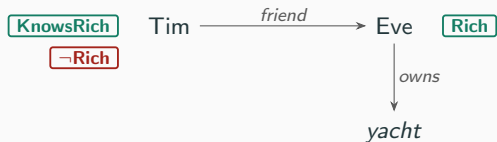
Rich

$\geq_1$ owns.yacht

KnowsRich

$\geq_1$ friend.Rich

Graph



The **labelling** is the **assignment** (what is checked to be true (false) so far).
Every validator computes one — on the call stack, or in a table.

Algorithm 1 — the validator everyone already has

```
 $\sigma$  : pairs (S, t) confirmed true so far
validate(S, t):
  if (S, t)  $\in \sigma$ : return true
  for each constraint needing S' at neighbour t':
    r := validate(S', t')
  result := combine the r's with S's local constraints
  if result:  $\sigma := \sigma \cup \{(S, t)\}$ 
  return result
for each target (S, t): validate(S, t)
```

Referencing, concretely

t_1 =Tim, s_1 =KnowsRich

via path=friend

$\rightarrow t_2$ =Eve, s_2 =Rich

call: validate(Rich, Eve)

σ , as it builds up: $\sigma_0 = \emptyset$; Eve owns a yacht $\Rightarrow \sigma_1 = \{(\text{Rich}, \text{Eve})\}$; Tim's friend Eve is Rich
 $\Rightarrow \sigma_2 = \sigma_1 \cup \{(\text{KnowsRich}, \text{Tim})\}$

Where it breaks

Shapes

Elite

$\geq_1 \text{ owns.yacht}$

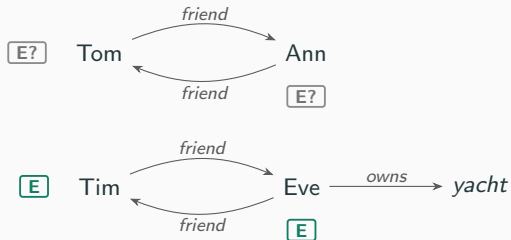
$\vee \geq_1 \text{ friend.Elite}$

Target

Elite(Tom), **Elite**(Ann),

Elite(Tim), **Elite**(Eve)

Graph



Eve: yacht. Tim: his friend Eve. But **Elite**(Tom) needs Ann, who needs Tom — **validate never returns**.

The fix: three lines

```
▶ validate(S, t, chain):  
▶   if (S, t) in chain: return DEFAULT  
    for each constraint of S needing shape S' at a neighbour t':  
▶     r := validate(S', t', chain ∪ {(S, t)})  
    return combine the r's with S's local constraints
```

- ▶ Three lines changed: an extra argument, **chain**, tracking the pairs above us on this call.
- DEFAULT = true $\Rightarrow$ **brave** (greatest fixed point); false $\Rightarrow$ **cautious** (least fixed point).
- σ is not gone — it only grows with pairs confirmed *outside* any chain.
- Corresponds to standard Prolog **SLD** resolution for LFP; i.e., **co-SLD** for GFP.

Two answers, both defensible

Cautious — least fixed point



Believe it only with a justification
that does not assume itself.

Brave — greatest fixed point



Accept any labelling that is
self-consistent.

Data **complete**? Be as restrictive as possible — **cautious**.

Data still **arriving**? Give it some slack — **brave**. (ShEx implements brave)

The fragment: positive SHACL

This works whenever the fragment is **monotone**: once a pair is in the assignment, adding more can never take it out.

Out — these negate a shape reference:

- `sh:not`
- `sh:qualifiedMaxCount`
- `sh:qualifiedValueShapesDisjoint`
- `sh:xone`

In — everything else in SHACL Core:

- `sh:minCount`, `sh:maxCount`
- `sh:qualifiedMinCount`
- `sh:node`, `sh:and`, `sh:or`
- `sh:closed`, datatypes, ...

Adding negation: stratification is NOT the ideal tool

Shapes

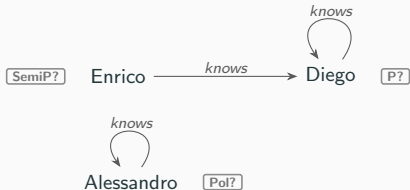
Polentone

$\forall \text{knows. Polentone}$

SemiPolentone

$\text{NOT } \geq_1 \text{ knows. Polentone}$

Graph



Target

SemiPolentone(Enrico), **Polentone**(Alessandro)

Diego *knows* himself — recursive. Brave **GFP** (what **ShEx** uses) **rejects** Enrico; LFP accepts him. Alessandro *knows* himself too: LFP **rejects** **Polentone**(Alessandro), GFP **accepts** it, so neither LFP nor GFP satisfy both targets.

⇒ If the goal is satisfy targets, even under stratified SHACL, neither LFP nor GFP is ideal, due to presence of negation

Supported Model Semantics: most general semantics

SuMS (Supported Model Semantics): **guess** an assignment σ such that

- σ satisfies the shape constraints;
- σ satisfies the targets.

Recall **Polentone**/**SemiPolentone**: Diego knows himself, Enrico knows Diego — target **SemiPolentone**(Enrico).

Three guesses for σ :



✗ inconsistent guess



■ shapes ok, target fails



● shapes & target ok

Algorithm 2 — checking a guessed σ (SuMS)

```
guess  $\sigma(s) \subseteq \text{dom}(G)$ , for each shape  $s$ 
for each shape  $s$ :
    eval( $s, \sigma$ ) := nodes where def( $s$ ) holds, taking  $\sigma$  over the neighbours
    if eval( $s, \sigma$ )  $\neq \sigma(s)$ : return NO      s is not self-consistent
for each target  $(s, t)$ :
    if  $t \notin \sigma(s)$ : return NO
return YES
```

- **Caveat 1:** finding a valid σ that also meets the targets can be **intractable** — NP-complete in general.
- **Caveat 2:** **no unique** σ — the earlier picture had two valid models; but only *one* valid wrt targets.

Implemented in two papers: **ISWC 2019** (SAT-based), **WWW 2020** (ASP-based).

The landscape

Fragment	Semantics	Values	Tractable	Extends the algorithm?
Non-recursive	—	2	yes	it <i>is</i> the algorithm
Positive recursion	cautious (LFP)	2	yes	yes — add chain
Positive recursion	brave (GFP)	2	yes	yes — same, other default
Stratified negation	stable models	2	yes	only with an outer loop
Stratified negation	supported models	2	only if <i>strictly</i> strat.	only with that condition
Unrestricted	supported	2	NP-complete	guess and check
Unrestricted	stable, brave	2	NP-complete	guess and check
Unrestricted	well-founded	3	yes	no — own construction

The first three rows come almost for free. The last three need a solver.

Conclusion

Positive recursion — easy to add, behaves well, extends the algorithm every implementation already has. One extra argument, one default.

Arbitrary SHACL — guess and check, a new algorithm, but not a hard one. Intractable in the worst case.

Three possible profiles:

1. Support none — status quo, but explicitly and by choice.
2. Support positive recursion — defined, both readings, tractable.
3. Full recursion over negation — solver-based, full expressiveness.

more at github.com/w3c/data-shapes/issues/1199

Thank you for the attention!

References

- Corman, Reutter, Savković. *Semantics and Validation of Recursive SHACL*. ISWC 2018.
- Corman, Reutter, Savković. *A Tractable Notion of Stratification for SHACL*. ISWC 2018 (P&D).
- Corman, Florenzano, Reutter, Savković. *Validating SHACL Constraints over a SPARQL Endpoint*. ISWC 2019.
- Andresel, Corman, Ortiz, Reutter, Savković, Šimkus. *Stable Model Semantics for Recursive SHACL*. WWW 2020.
- Ahmetaj, Löhnert, Ortiz, Šimkus. *Magic Shapes for SHACL Validation*. VLDB 2022.
- Okulmus, Šimkus. *SHACL Validation under the Well-founded Semantics*. KR 2024.
- Ahmetaj et al. *Common Foundations for SHACL, ShEx, and PG-Schema*. WWW 2025.
- Ahmetaj et al. *Common Foundations for Recursive Shape Languages*. KR 2026.
- Boneva, Labra Gayo, Prud'hommeaux. *Semantics and Validation of Shapes Schemas for RDF*. ISWC 2017.